Gerência de Memória

Professor: Paulo Roberto de Oliveira
Disciplina: Sistemas Operacionais

Departamento de Informática
Universidade Estadual de Maringá

Introdução

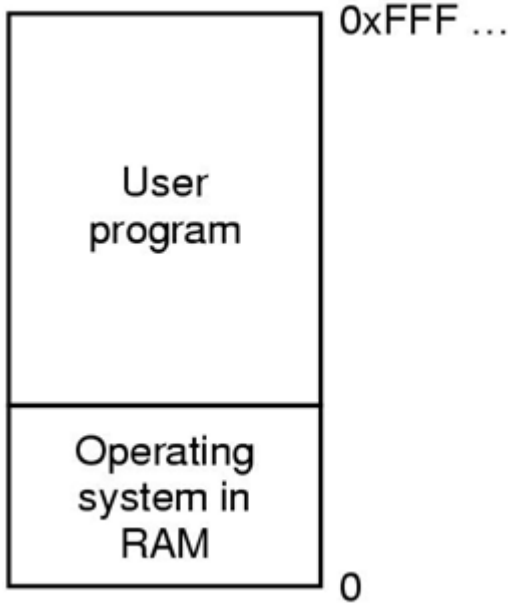
- **Lei de Parkinson:** Os programas crescem até ocupar toda a memória disponível.
- **Gerente de Memória:**
  - Módulo do sistema operacional que gerencia a memória principal e a área de memória secundária associada à memória principal;
  - Controla as áreas de memória em uso;
  - Controla as áreas de memória disponíveis;
  - Cede memória aos processos quando eles necessitam;
  - Retorna áreas dos processos para a memória disponível quando elas não são mais necessárias;
  - Realiza swapping.

Swapping

- Normalmente há mais usuários do que memória para manter os programas necessários!
- Processos interativos em geral ficam a maior parte do tempo parados à espera de requisições dos usuários;
- Os processos excedentes devem ficar em disco;
- Para executar (atender uma requisição), o processo precisa estar em memória;
- É preciso fazer movimentação memória $\longleftrightarrow$ disco;
- Esta movimentação é chamada de **Swapping**.

Monoprogramação
Sem Swaping ou Paging

- Um processo apenas por vez;
- Este processo usa toda a memória disponível;
- Método em desuso!



Multiprogramação e Uso de Memória

- Razões para se usar multiprogramação:
  - Atende mais usuários ao mesmo tempo;
  - Utiliza melhor os recursos do sistema (como a CPU).
  - Em geral, um programa passa a maior parte do tempo esperando pelo término de operações de I/O;
- **Exemplo**: Se um processo usa apenas 20% de seu quantum, com monoprogramação 80% da CPU seria desperdiçado;

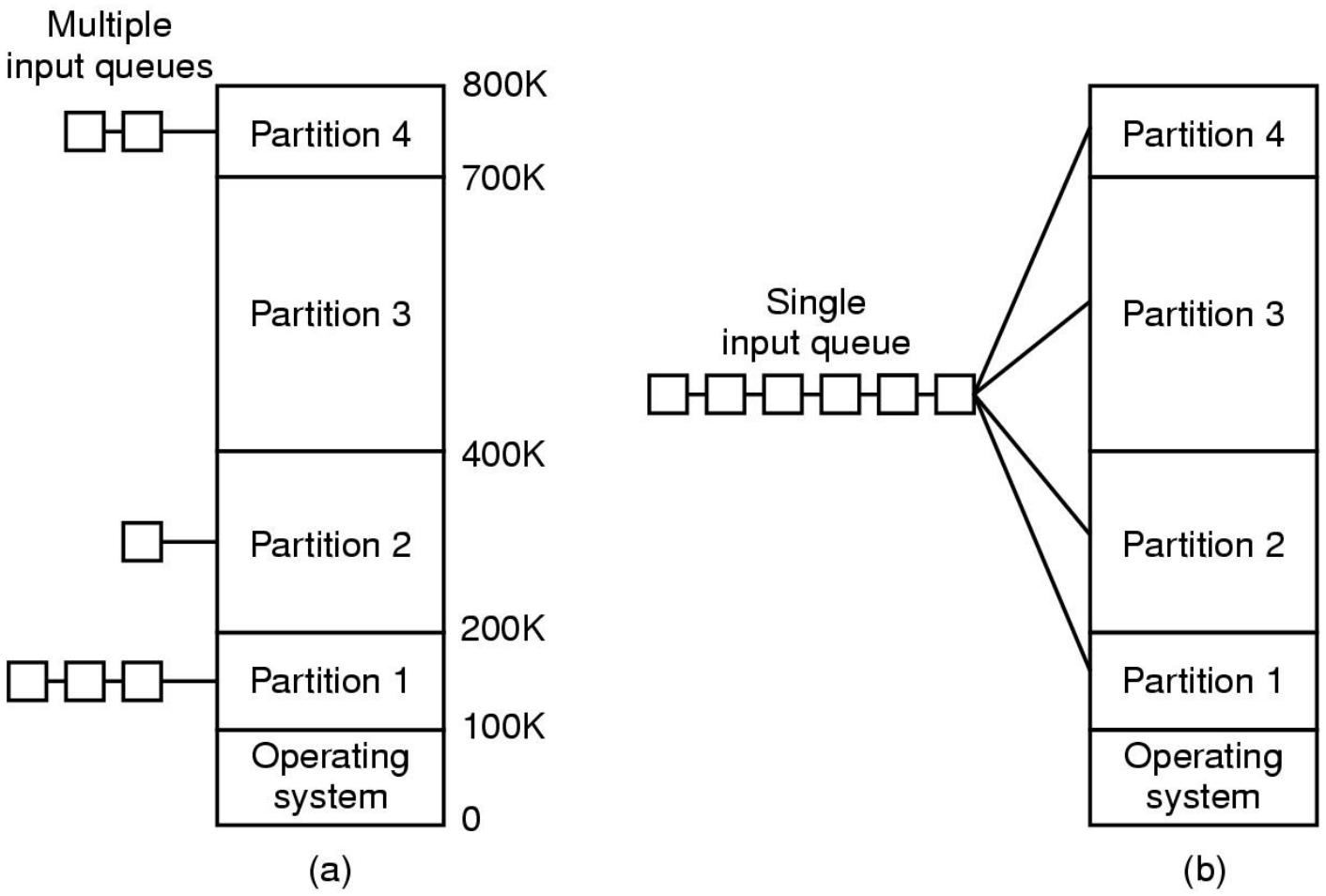
Multiprogramação com partições fixas

- Ter mais de um processo em memória executando simultaneamente é muito útil. A questão é **como organizar a memória**.
- A primeira solução é dividir a memória em n áreas fixas e em cada uma colocar um processo;
- Essas áreas são chamadas de **partições**;
- O tamanho das partições não precisa e nem deve ser igual;
- O espaço não usado de uma partição é perdido;

Multiprogramação com partições fixas

- O processos são organizados em filas para usar as partições;
- Desvantagem de se usar múltiplas filas para as partições:
  - A maioria dos processos são pequenos;
  - As filas para partições grandes podem ficar vazias enquanto as das pequenas ficam cheias, com jobs que não podem começar por falta de partições adequadas;
- Uma alternativa é ter uma fila única para todas as partições;
- Neste caso, toda vez que uma partição fica livre o sistema procura na fila o maior job que cabe nessa partição;

Multiprogramação com partições fixas



Multiprogramação com partições fixas

- O método de fila única dá preferência para os procesoss maiores;
- **Estratégia:** Toda vez que um processo do início da fila for preterido por outro maior, ele recebe um ponto. Quando atingir **k** pontos, ele não pode mais ser preterido.

Relocação e Proteção

- Diferentes processos são carregados em diferentes endereços de memória (diferentes partições);
- De uma execução para outra, o processo pode mudar de partição;

Relocação e Proteção

- **Exemplo:** A 1ª instrução de um programa é uma chamada de um procedimento no endereço 100.
- Se o processo é carregado na Partição 1 (começa em 100 KB), o endereço 100 fica dentro do SO (fora da partição do processo!);
- Desta forma, o endereço deveria ser 100 Kb + 100;
- Se o processo é carregado na partição 2, o endereço 100 deve ser na verdade 200 Kb + 100;
- Este sistema é chamado de **Problema de Relocação**;

Relocação e Proteção

- **Uma solução**: Alterar todas as instruções somando o endereço base!
- **Problema**: É preciso identificar quais instruções têm endereços
- O *linker* precisa colocar no código uma tabela dizendo quais instruções precisam ser alteradas.

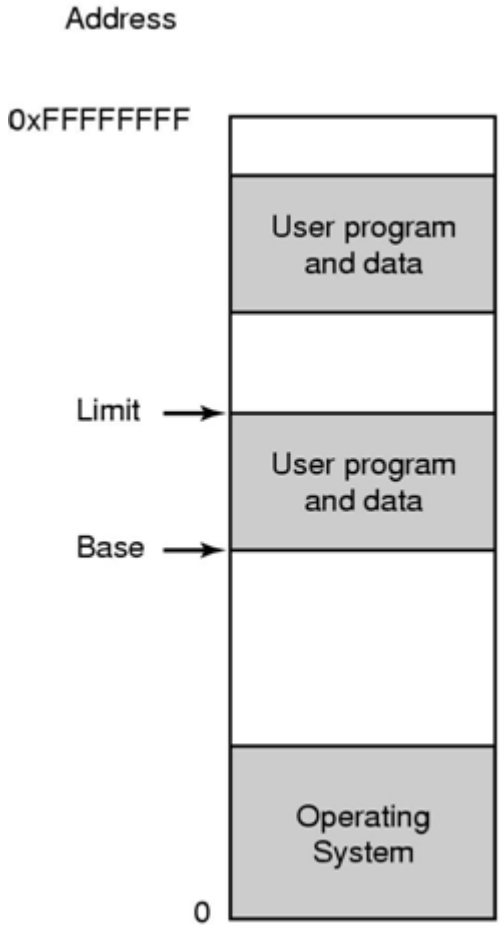
Relocação e Proteção

- Programas podem dar jumps usando instruções como ponteiros;
- Assim, podem acessar endereços fora de suas partições!
- Isso não é aceitável em ambientes de multiprogramação e, principalmente, multiusuários!

Relocação e Proteção

- **Solução:** Registradores Base e Limite.
- Quando um processo é escalonado, o registrador base é carregado com o endereço base da partição e o limite com o endereço final;
- Todo acesso à memória tem o endereço somado ao endereço base;
- O resultado deve ser menor do que o endereço limite;
- **Vantagem:** O programa pode ser relocado depois de iniciada sua execução!

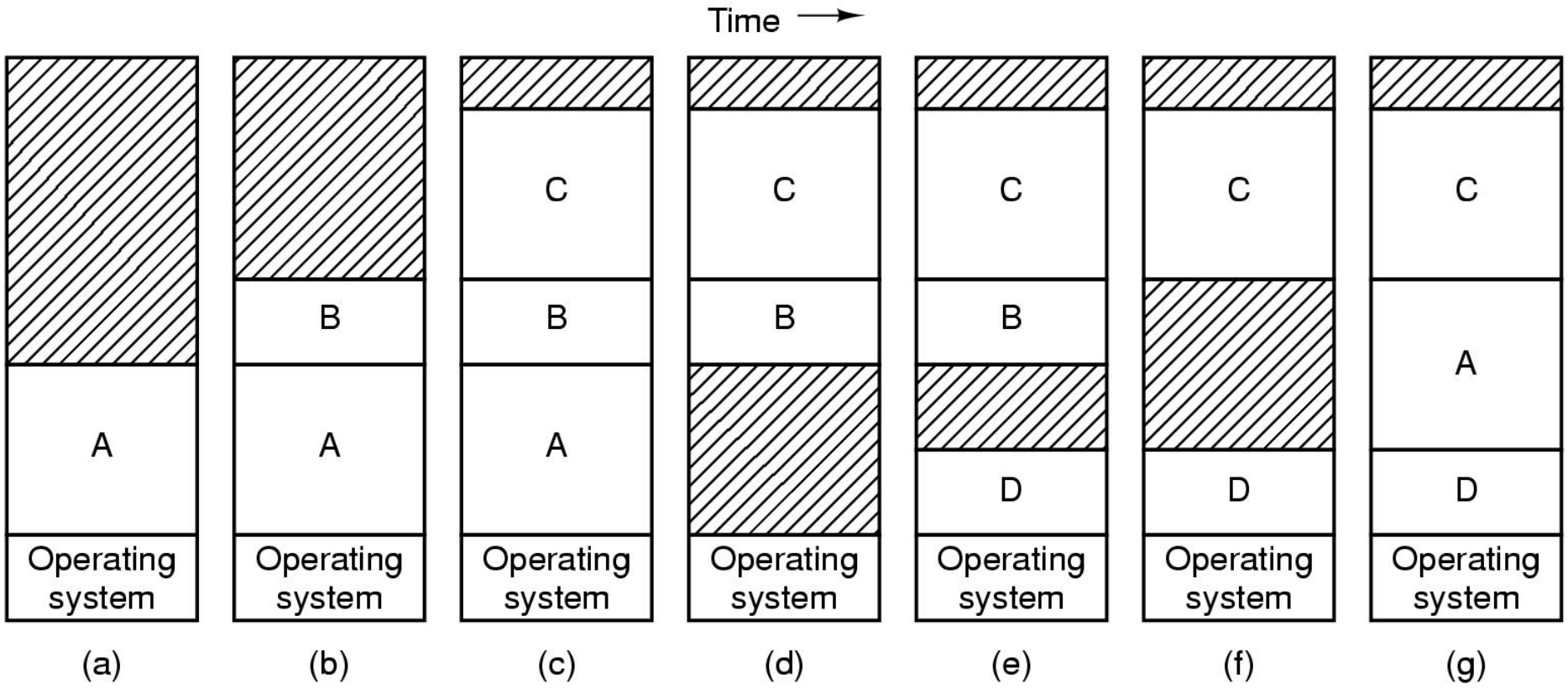
Relocação e Proteção



Multiprogramação com Partições Variáveis

- Usado em sistemas bastante dinâmicos (número e tamanho dos processos em memória varia);
- Este método melhora a utilização de memória, mas complica a alocação e desalocação de áreas e também o gerenciamento das áreas em uso;
- É possível agrupar as áreas disponíveis movendo todos os processos para baixo (compactação de memória);
- **Obs.:** Gasta muita CPU copiar áreas de memória;

Multiprogramação com Partições Variáveis



Multiprogramação com Partições Variáveis

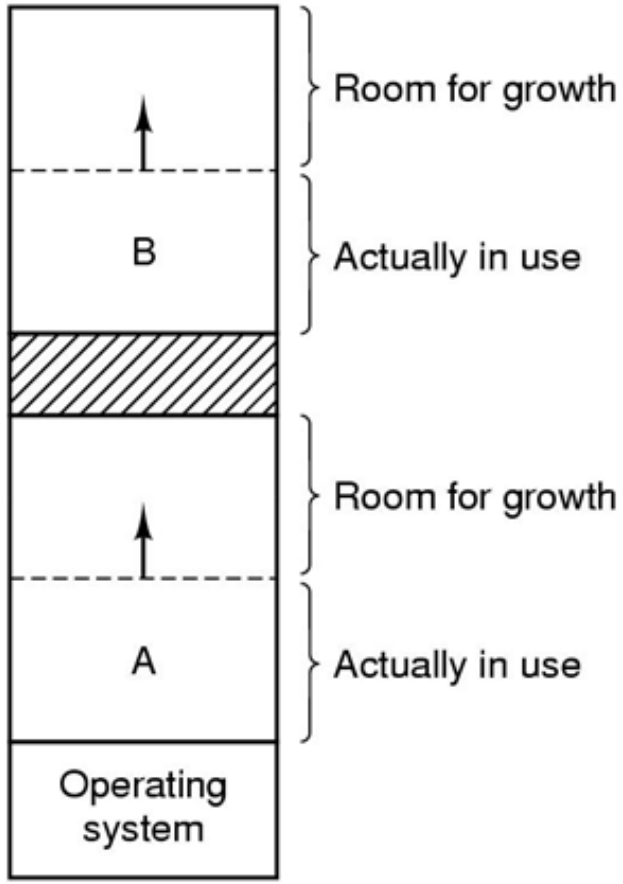
- Quanta memória um processo precisa?
  - Se todos os processos tiverem tamanho fixo, não há problemas;
  - Basta dar a memória que eles precisam e pronto.

- Muitas linguagens utilizam heaps (espaços de memória especiais) de onde os processos retiram áreas de que precisam dinamicamente e acrescentam-nas em seus segmentos de dados.

Multiprogramação com Partições Variáveis

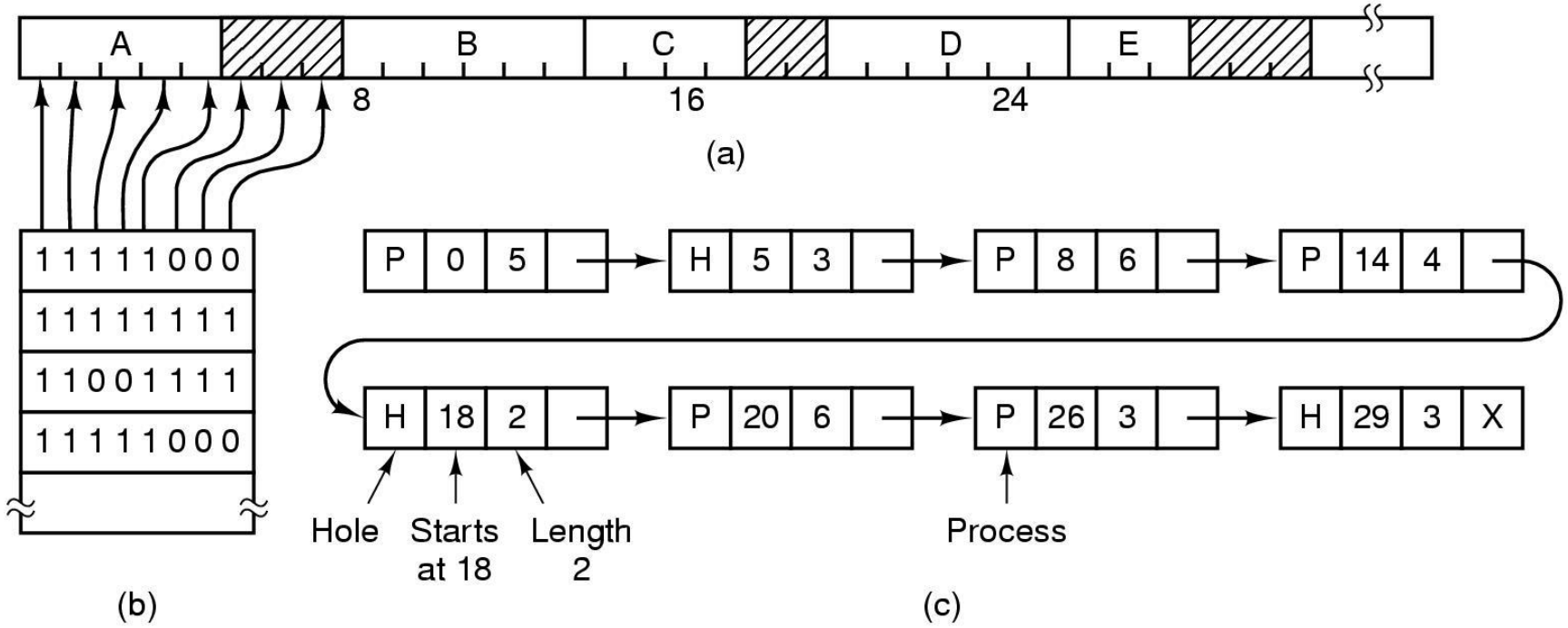
- O que acontece se o processo tenta crescer?
- Se há espaço na frente, ele pode simplesmente ocupar essa área;
- Se houver um processo na frente, há 2 opções:
  - Ele pode ser movido para um buraco grande o suficiente para conter o novo tamanho do processo;
  - Um ou mais processos podem ser movidos para disco para liberar espaço.
- Se o processo não pode crescer e a área de swap em disco está cheia, o processo fica bloqueado ou é exterminado.

Multiprogramação com Partições Variáveis



Gerencia de Memória com Bit Maps

- Memória dividida em unidades de alocação (poucas palavras até vários KB);



Gerencia de Memória com Bit Maps

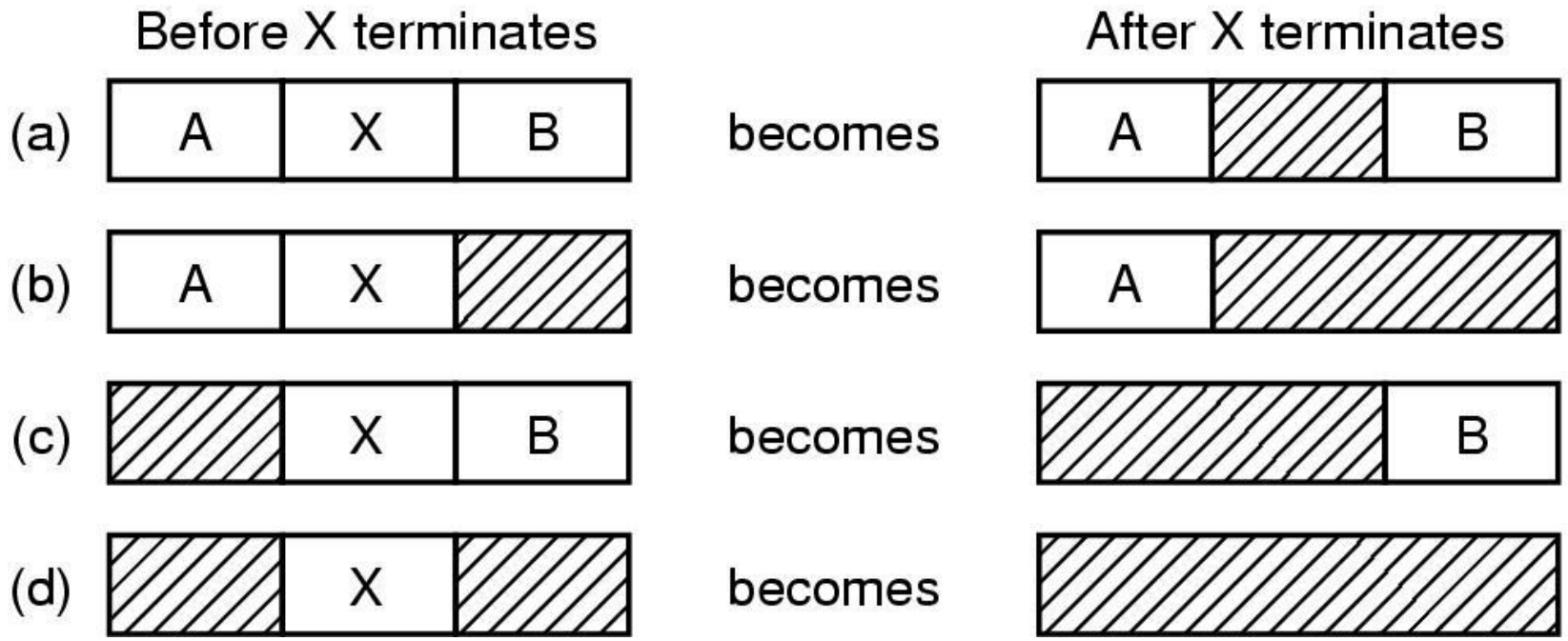
- Questões que devem ser consideradas:
  - Qual o tamanho da unidade?
  - Se ela é pequena (ex.: 4 bytes), a tabela de alocação é grande;
  - Se a unidade é grande (ex.: 1 MB), a tabela é pequena, em compensação, o desperdício com processos pequenos é grande;
  - Quanto maior a unidade, menos operações de alocação são necessárias para dar o que os processos querem.

Gerencia de Memória com Listas Encadeadas

- Organiza a memória em listas de segmentos;
- Um segmento é um processo ou é um buraco entre 2 processos;
- O controle de processos e buracos pode ser separado, ou na mesma lista;
- Em geral, um processo X pode estar entre 2 processos ou entre 2 buracos;

Gerencia de Memória com Listas Encadeadas

- Quando X termina, há 4 possibilidades:



Gerencia de Memória com Listas Encadeadas

- Considere que um processo solicita uma área de memória.
- Existem alguns algoritmos de pesquisa para listas encadeadas classificadas por endereço:

- **First Fit:** O sistema escolhe a primeira área grande o suficiente para atender a requisição;
  - A área é dividida em 2: processo e sobra;
  - Tende a não varrer a memória inteira, demorando para encontrar defeitos;
  - Gera grandes espaços.

Gerencia de Memória com Listas Encadeadas

- **Next Fit:** Semelhante ao anterior, mas inicia a pesquisa de onde parou na última vez.

- **Best Fit:** Procura na lista inteira pela área mais adequada para atender a requisição;
  - Mais lento;
  - Testes mostram que desperdiça mais memória – tende a criar muitos espaços pequenos.

- **Worst Fit:** Procura pelo maior espaço que atende a requisição;
  - Testes mostram que não é bom.

Gerencia de Memória com Listas Encadeadas

- **Melhoria:** Separar processos e espaços:
  - Só procura nos espaços – evita as áreas ocupadas;
  - É mais demorado quando um processo termina;
  - Ordenação dos espaços por tamanho – é melhor para os métodos best fit, first fit e next fit;

- **Quick Fit:** Listas separadas por diversos tamanhos.

Gerencia de Memória com Listas Encadeadas

- **Problema do Quick Fit:** Quando um processo termina, é difícil achar os blocos vizinhos.
- Localizar o vizinho para tentar um merge é caro;
- Sem o merge, a memória fica fragmentada em grande número de espaços pequenos e sem utilidade.

Gerencia de Memória com o Sistema Buddy

- Tira vantagem de que os computadores usam números binários para o endereçamento.
- O Gerente de Memória mantém uma lista dos blocos livres para cada tamanho múltiplo de 2: 1, 2, 4, 8, 16, ..., 1024k bytes.
- Quando começa a atividade, o sistema está com toda a memória livre. Há apenas 1 entrada na lista de 1 MB; todas as outras listas estão vazias.

Gerencia de Memória com o Sistema Buddy

Inicialmente:

1 MB	

Requisita 70 KB p/ A

A	128	256 KB	512 KB

Requisita 35 KB p/ B:

A	B	64	256 KB	512 KB

Requisita 80 p/ C:

A	B	64	C	128	512 KB

Retorna A:

128	B	64	C	128	512 KB

Requisita 60 p/ D:

128	B	D	C	128	512 KB

Retorna B:

128	64	D	C	128	512 KB

Retorna D:

256	C	128	512 KB

Retorna C:

1 MB

Gerencia de Memória com o Sistema Buddy

- **Vantagens:**
  - O sistema Buddy é bastante rápido, pois usa tamanhos de blocos múltiplos de 2 (base do computador).

- **Problemas:**
  - É ineficiente!
  - Uma área de 35 KB precisa de 64 KB – 29 KB são perdidos;
  - Apresenta fragmentação interna – áreas alocadas para os processos e não utilizadas;
  - Apresenta fragmentação externa – buracos entre os processos.

Alocação do espaço Swap

- Opções de Swap Space:
  - Nenhum espaço em disco é alocado para um processo em memória;
    - Quando ele precisa deixar a memória, uma área de swap precisa ser alocada;
    - A cada vez, a área será num lugar diferente.
  - Alocação de área em disco quando o processo é criado;
    - Toda vez que ele sofre *swap out*, ele vai para a mesma área;
    - Essa área é desalocada quando o processo termina.

Dúvidas?